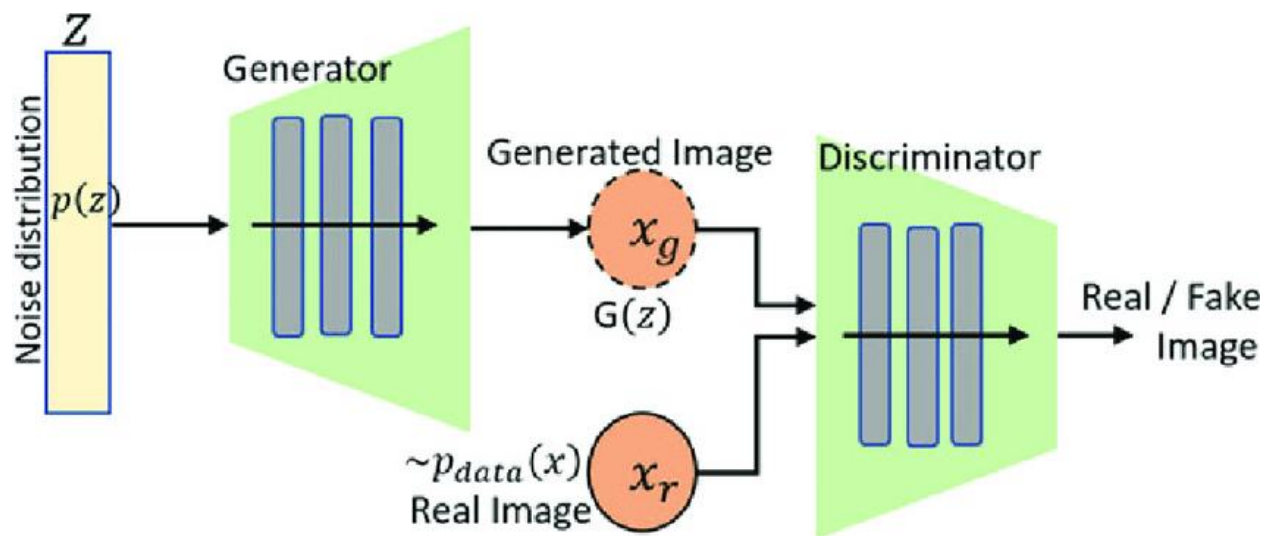


Standard GAN (Vanilla GAN)



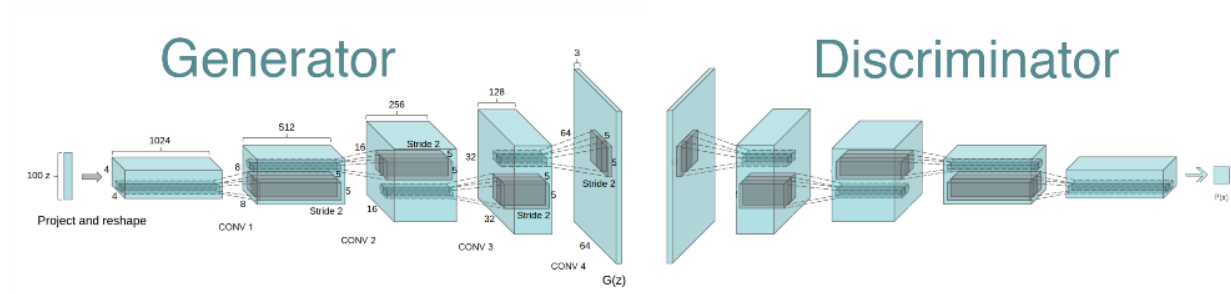
GAN(Multi layer perceptron)



DCGAN

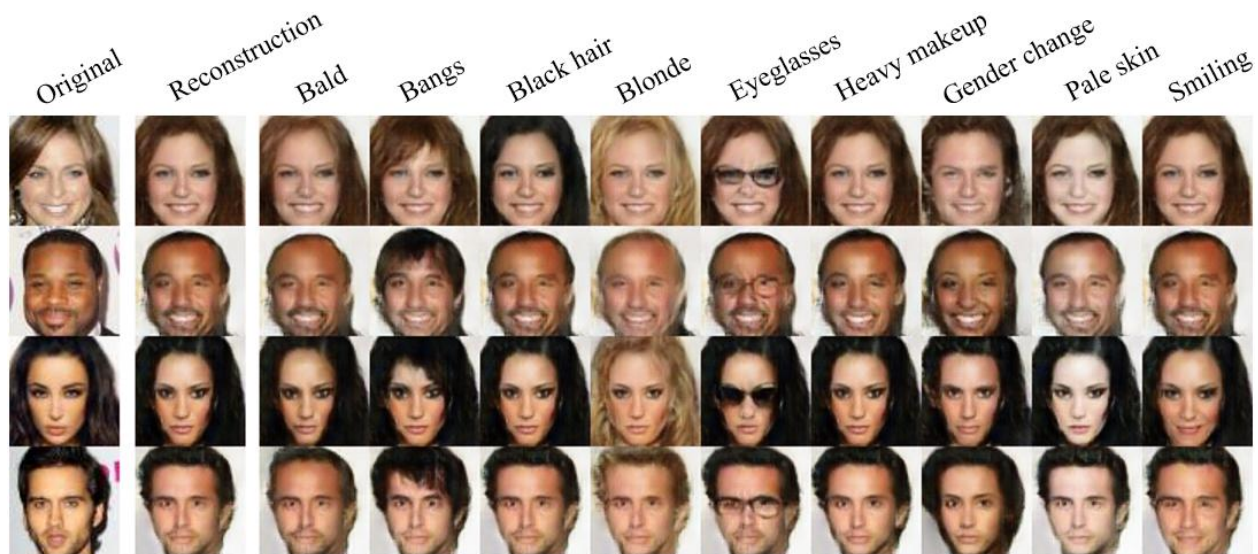
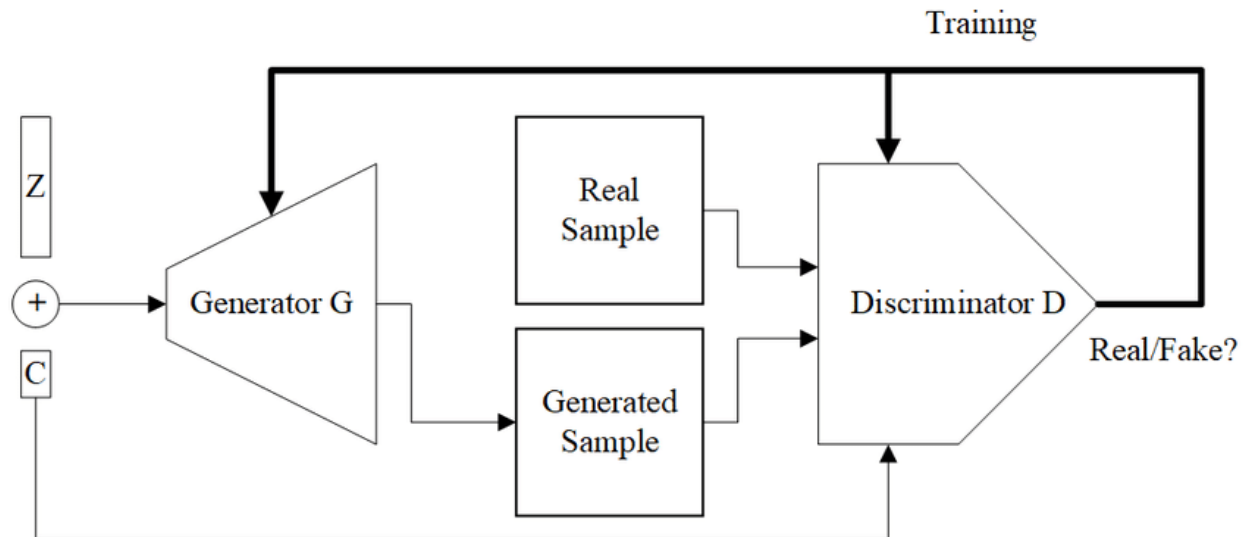


Deep Convolutional GAN (DCGAN)



Conditional GAN (cGAN)

The input noise z and the output of D are conditioned on some extra information, c (e.g., class labels).



Wasserstein GAN (WGAN)

Critic (D) outputs real numbers, not probabilities. Replaced the JS Divergence loss with the **Earth Mover's Distance** (Wasserstein distance)

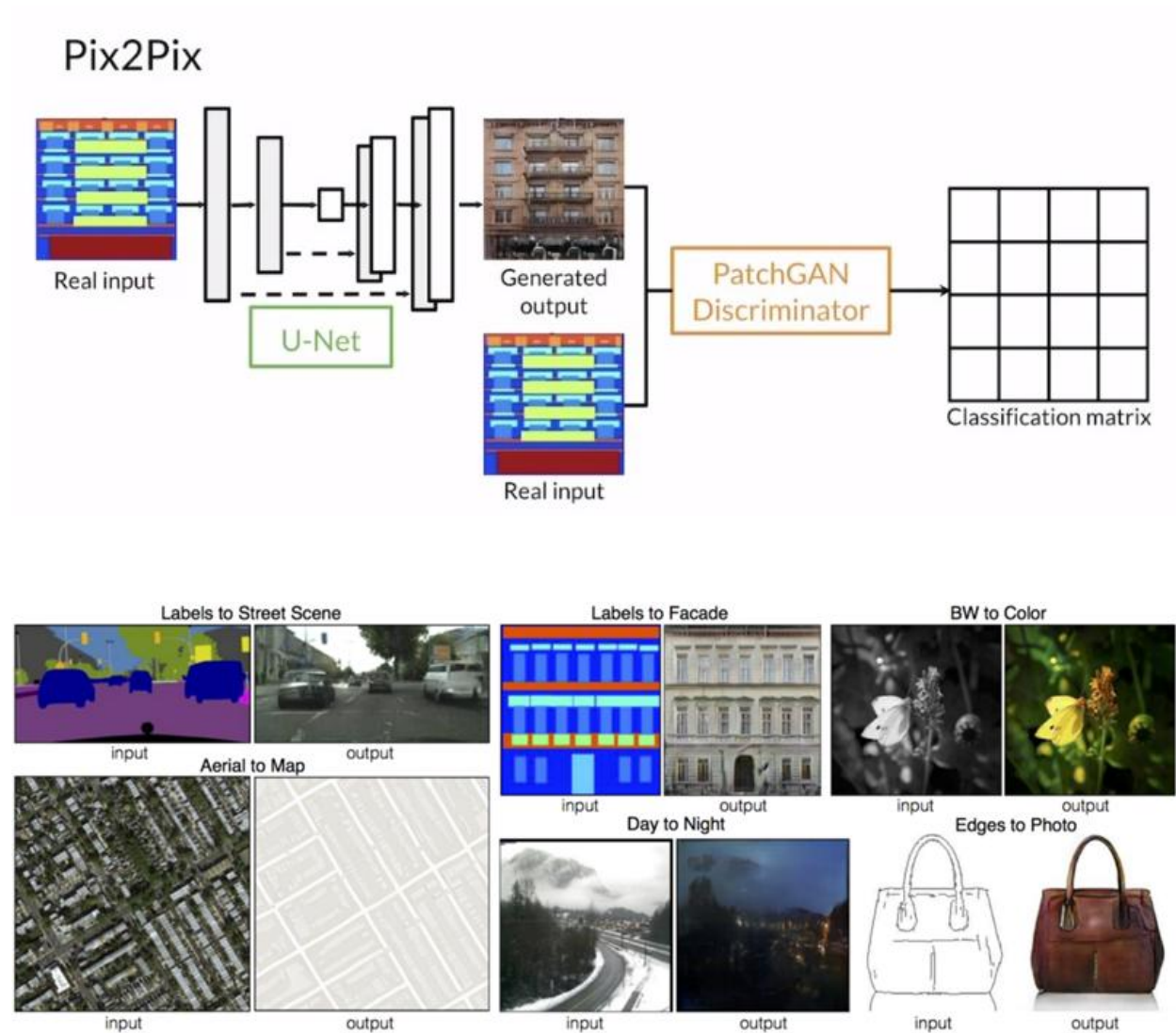
$$\text{GAN: } \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)})))]$$

$$\text{WGAN: } \nabla_w \frac{1}{m} \sum_{i=1}^m [f(\mathbf{x}^{(i)}) - f(G(\mathbf{z}^{(i)}))]$$



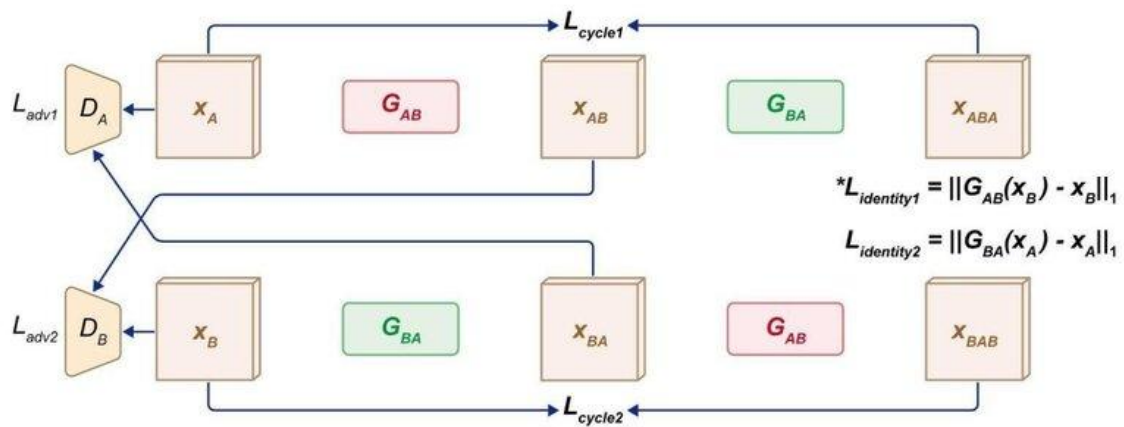
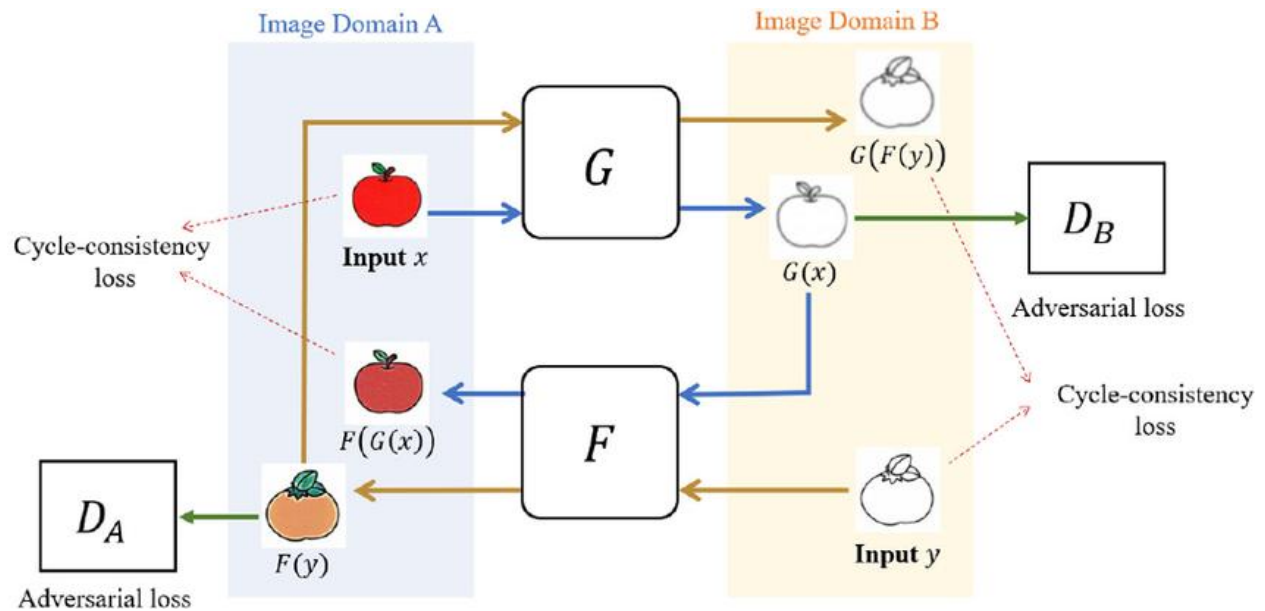
Pix2Pix (Image-to-Image)

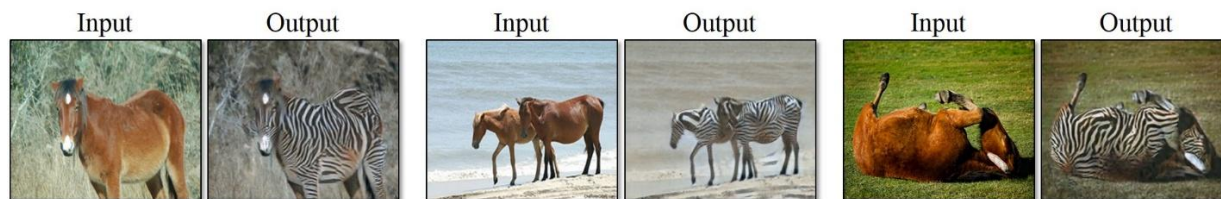
A cGAN that conditions the generation on a full input image, using a U-Net architecture for the Generator



CycleGAN

Uses a cycle-consistency loss to learn the mapping between two domains (A $\rightarrow$ B and B $\rightarrow$ A) *without* requiring paired training data.





horse → zebra



zebra → horse



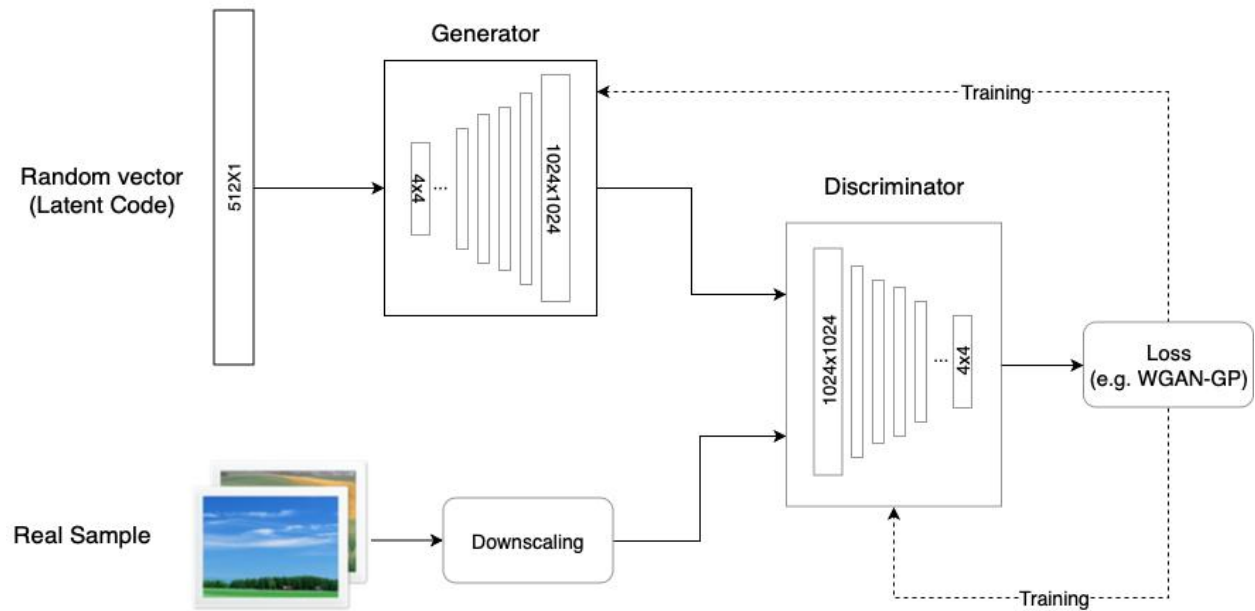
apple → orange



orange → apple

ProGAN (Progressive Growing GAN)

Starts training with very low-resolution images and progressively adds new layers to both the G and D as training advances. Uses Bi-Linear Sampling.



a

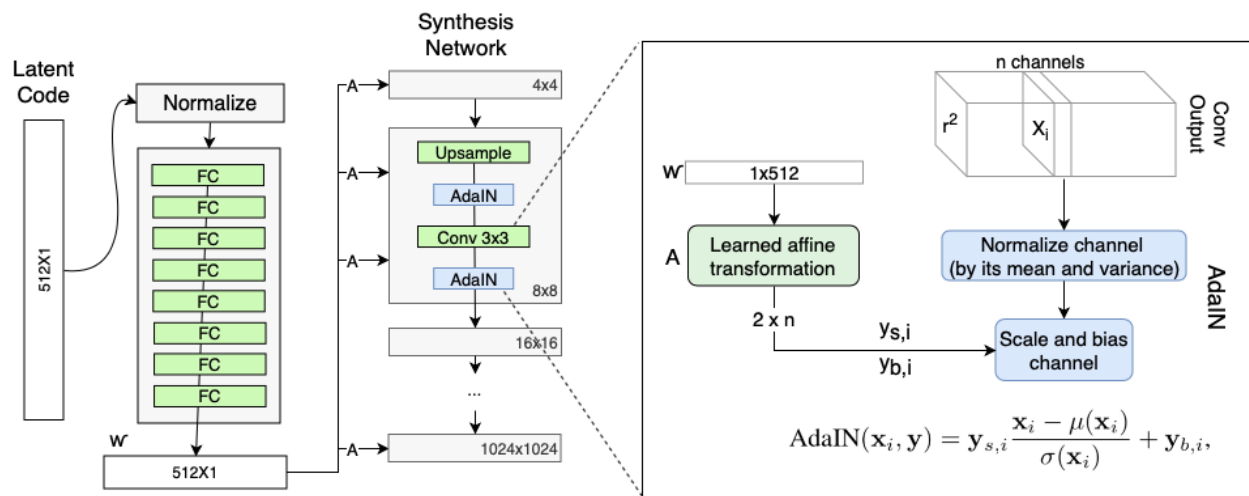


b

StyleGAN

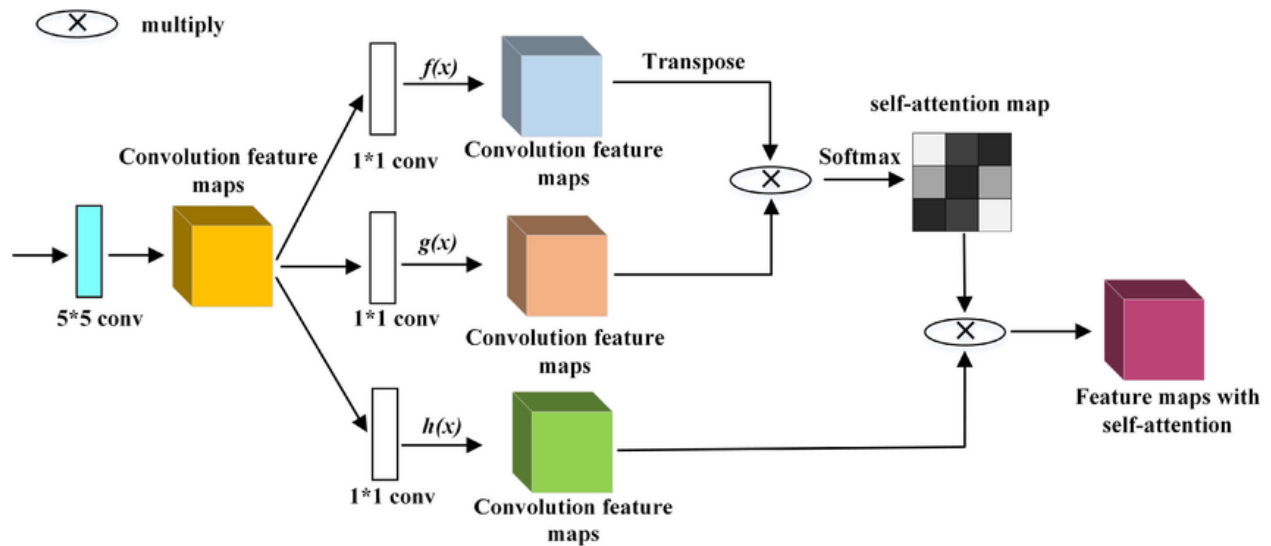
Introduced style-based architecture where latent code is processed by a mapping network and fed into the Generator at multiple scales via adaptive instance normalization (AdaIN). Progressive growing+ Bilinear Sampling+Mapping Network+AdaIN.

Adaptive Instance Normalization aligns the statistics of one feature map to another. StyleGAN uses it to inject style information into each convolution block.



Self-Attention GAN (SAGAN)

Incorporated attention mechanisms into G and D, allowing them to focus on globally consistent details regardless of distance in the image.



StackGAN

Generates high-resolution images from text descriptions by using a multi-stage process: first generating a low-res image, then refining it in subsequent stages.

